EX 9 TEXT GENERATION USING LSTM NETWORKS DATE:

Problem Statement:

Build a text generation model using Long Short-Term Memory (LSTM) networks. Train the model on a text corpus to generate coherent sequences of text and evaluate the output for fluency and coherence.

Suggested Dataset: Shakespeare Corpus

Objectives:

1. Understand sequential modeling for natural language generation.
2. Train a character-level LSTM model to learn language patterns.
3. Generate text using a seed prompt and evaluate the results.
4. Analyze the fluency and creativity of LSTM-generated outputs.

Scope:

Text generation is a foundational task in natural language processing. This experiment demonstrates how LSTMs can learn syntactic and semantic patterns over time and generate believable sequences of text. The use of character-level modeling helps capture detailed language structures.

Tools and Libraries Used:

1. Python 3.x
2. TensorFlow / Keras
3. NumPy
4. Shakespeare Text Corpus (Tiny Shakespeare)

Implementation Steps:

Step 1: Load and Preprocess the Dataset

```
import tensorflow as tf
import numpy as np

text = tf.keras.utils.get_file('shakespeare.txt',
 'https://raw.githubusercontent.com/karpathy/char
rnn/master/data/tinyshakespeare/input.txt')
text = open(text, 'r').read().lower()
chars = sorted(set(text))
c2i = {c: i for i, c in enumerate(chars)}
i2c = {i: c for i, c in enumerate(chars)}
```

Step 2: Create Input and Output Sequences

```
seq_len = 40
X = []
y = []

for i in range(len(text) - seq_len):
 input_seq = text[i:i + seq_len]
 target_char = text[i + seq_len]
 X.append([c2i[c] for c in input_seq])
 y.append(c2i[target_char])

X = np.array(X)
y = np.array(y)
```

Step 3: Build the LSTM Model

```
model = tf.keras.Sequential([
 tf.keras.layers.Embedding(len(chars), 64, input_length=seq_len),
tf.keras.layers.LSTM(128),
 tf.keras.layers.Dense(len(chars), activation='softmax')
])
model.compile(loss='sparse_categorical_crossentropy',
optimizer='adam') model.fit(X, y, batch_size=128, epochs=1)
```

Step 4: Define the Text Generation Function

```
def generate(seed, length=300):
 seq = [c2i[c] for c in seed.lower()]
 for _ in range(length):
 inp = np.array(seq[-seq_len:]).reshape(1, -1)
 pred = model.predict(inp, verbose=0)[0]
 next_idx = np.random.choice(len(pred), p=pred)
 seq.append(next_idx)
 return seed + ''.join(i2c[i] for i in seq[len(seed):])
```

Step 5: Generate and Display Text

```
print("\nGenerated Text:\n")
print(generate("shall i compare thee to a summer's day?\n"))
```

Conclusion:

This experiment demonstrates how an LSTM model learns to predict and generate sequences of text based on character-level inputs. Despite being trained for a single epoch, the generated output showcases structural fluency and stylistic hints of Shakespearean English. Further training can improve coherence and creativity.

SAMPLE EXPECTED OUTPUT

Experiment 9: Text Generation using LSTM

Character-level text generation from Tiny Shakespeare

Illustrative only. The experiment was not executed. All example values, curves and text below are invented to show the expected output format; they are not measured results.

Sample console / output format

```
Generated Text:
shall i compare thee to a summer
the king and i the worl of thee,
and in the night the heart shall be
to speake the day and all the light.
```

Output element	Expected form
Training progress	One epoch of character prediction loss
Generated continuation	300 sampled characters after the seed
Output variability	Different continuations across runs

How to read this output

The text above is a shortened, authored illustration, not an LSTM result. One training epoch can yield misspellings, repetition and broken grammar. The manual seed has 38 characters but seq_len is 40: provide at least 40 seed characters or pad the input before generation.